```
#!/usr/bin/env python3
# ================================================================
# primecount benchmark -- verify the reported numbers yourself
# ================================================================
# Measures:
#   1. the actual nth_prime limit (the function the model depends on)
#   2. the prime_pi limit
#   3. per-call cost at different scales
#   4. the true values used in every experiment
#
# Every number is measured on your machine, not quoted from elsewhere.
# ================================================================

import time, sys, math

try:
    from primecountpy import primecount as pc
except ImportError:
    import subprocess
    print("primecount not found -- installing...")
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "primecountpy"]
                   check=False)
    try:
        from primecountpy import primecount as pc
    except ImportError:
        print("install failed. Run manually: pip install primecountpy")
        sys.exit(1)

print("=" * 66)
print("1. Available functions")
print("=" * 66)
print("  ", [x for x in dir(pc) if not x.startswith("_")])

# ================================================================
print()
print("=" * 66)
print("2. nth_prime limit  (this is what bounds the model)")
print("=" * 66)
# [important] primecount calls terminate() when its limit is exceeded --
# killing the process rather than raising a catchable Python exception.
# So we probe in a **separate process** and read the limit from stderr.
LIMIT_NTH = None
import subprocess, re
_probe = subprocess.run(
```

```
    [sys.executable, "-c",
     "from primecountpy import primecount as pc; pc.nth_prime(10**18)"],
    capture_output=True, text=True, timeout=120)
_m = re.search(r"<=\s*(\d+)", _probe.stderr + _probe.stdout)
if _m:
    LIMIT_NTH = int(_m.group(1))
else:
    # limit not stated: probe by powers of ten, each in its own process
    LIMIT_NTH = 10**6
    for _e in range(7, 19):
        _r = subprocess.run(
            [sys.executable, "-c",
             f"from primecountpy import primecount as pc; pc.nth_prime(10**{_e})"
            capture_output=True, text=True, timeout=300)
        if _r.returncode != 0: break
        LIMIT_NTH = 10**_e

print(f"  limit: n <= {LIMIT_NTH:,}")
print(f"              ≈ 10^{math.log10(LIMIT_NTH):.2f}")
print()
print("  direct test around the limit (each in a separate process, since\n  excee
for _n, _lbl in [(LIMIT_NTH, "at the limit"), (LIMIT_NTH + 1, "one above")]:
    _r = subprocess.run(
        [sys.executable, "-c",
         f"from primecountpy import primecount as pc; print(pc.nth_prime({_n}))"]
        capture_output=True, text=True, timeout=600)
    if _r.returncode == 0:
        print(f"    {_lbl:>16}: ok    -> {int(_r.stdout.strip()):,}")
    else:
        _em = re.search(r"what\(\):\s*(.+)", _r.stderr)
        print(f"    {_lbl:>16}: fail  -> {(_em.group(1) if _em else _r.stderr.str

# ============================================================
print()
print("=" * 66)
print("3. prime_pi limit")
print("=" * 66)
PI_TIMEOUT = 45      # per-call timeout (seconds). Raise it to reach further.
print(f"  (per-call timeout: {PI_TIMEOUT}s -- raise PI_TIMEOUT to go further)")
for e in [15, 18, 20, 22, 25, 28, 31]:
    t0 = time.time()
    try:
        r = subprocess.run(
            [sys.executable, "-c",
             f"from primecountpy import primecount as pc; print(pc.prime_pi(10**{
            capture_output=True, text=True, timeout=PI_TIMEOUT)
    except subprocess.TimeoutExpired:
```

```
        print(f"  pi(10^{e:<2}): timed out ({PI_TIMEOUT}s) -- feasible but slower
        break
    t = time.time() - t0
    if r.returncode == 0 and r.stdout.strip():
        print(f"  pi(10^{e:<2}) = {int(r.stdout.strip()):>28,}   in {t:>7.2f}s")
    else:
        em = re.search(r"what\(\):\s*(.+)", r.stderr)
        print(f"  pi(10^{e:<2}): fail -> {(em.group(1) if em else r.stderr.strip(
        break

# ============================================================
print()
print("=" * 66)
print("4. nth_prime cost by scale  (this sets the model calibration time)")
print("=" * 66)
NTH_TIMEOUT = 45     # per-call timeout (seconds). Raise it to reach further.
print(f"  (per-call timeout: {NTH_TIMEOUT}s)")
print(f"  {'n':>10} {'p(n)':>26} {'time':>10} {'500 calls':>12} {'2,000 calls':>1
print("  " + "-" * 74)
for e in [9, 10, 11, 12, 13, 14, 15, 16, 17]:
    n = 10**e
    if n > LIMIT_NTH:
        print(f"  10^{e:<8}  -- above the limit --")
        continue
    t0 = time.time()
    try:
        r = subprocess.run(
            [sys.executable, "-c",
             f"from primecountpy import primecount as pc; print(pc.nth_prime({n})
            capture_output=True, text=True, timeout=NTH_TIMEOUT)
    except subprocess.TimeoutExpired:
        print(f"  10^{e:<8}  timed out ({NTH_TIMEOUT}s) -- cost doubles rapidly h
        continue
    t = time.time() - t0
    if r.returncode != 0 or not r.stdout.strip():
        em = re.search(r"what\(\):\s*(.+)", r.stderr)
        print(f"  10^{e:<8}  fail: {(em.group(1) if em else '')[:45]}"); continue
    v = int(r.stdout.strip())
    print(f"  10^{e:<8} {v:>26,} {t*1000:>8.0f}ms "
          f"{t*500/60:>9.1f}min {t*2000/3600:>10.2f}h")

# ============================================================
print()
print("=" * 66)
print("5. Verify the true values used in the experiments")
print("=" * 66)
CHECKS = [
```

```
    (100_000_000,       2_038_074_743,      "T-base: n=10^8"),
    (1_000_000_000,     22_801_763_489,     "T1: n=10^9"),
    (2_000_000_000,     47_055_833_459,     "T2: n=2×10^9"),
    (1_000_000_000_000, 29_996_224_275_833, "T3: n=10^12"),
    (2**41,             67_756_520_645_329, "T4b: n=2^41"),
]
for n, expected, lbl in CHECKS:
    if n > LIMIT_NTH:
        print(f"  {lbl:>16}: above nth_prime limit -- cannot verify"); continue
    t0 = time.time()
    try:
        r = subprocess.run(
            [sys.executable, "-c",
             f"from primecountpy import primecount as pc; print(pc.nth_prime({n})
            capture_output=True, text=True, timeout=300)
    except subprocess.TimeoutExpired:
        print(f"  {lbl:>16}: timed out"); continue
    t = time.time() - t0
    if r.returncode != 0 or not r.stdout.strip():
        print(f"  {lbl:>16}: failed"); continue
    v = int(r.stdout.strip())
    ok = "ok" if v == expected else "MISMATCH!"
    print(f"  {lbl:>16}: {v:>20,}  {ok}   ({t:.1f}s)")
    if v != expected:
        print(f"  {'':>16}  expected: {expected:,}")

# ═══════════════════════════════════════════════════════════════
print()
print("=" * 66)
print("6. Summary")
print("=" * 66)
print(f"  nth_prime limit: {LIMIT_NTH:,}  ~ 10^{math.log10(LIMIT_NTH):.1f}")
print(f"  => the model cannot exceed this scale, because building the")
print(f"     calibration grid depends on nth_prime.")
print()
print(f"  The prime_pi limit is far higher (see section 3).")
print(f"  => rebuilding the grid using pi(x) instead of nth_prime would")
print(f"     raise the model ceiling. This change is not implemented.")
print("=" * 66)
```